ECOPAC GUATEMALA · ECOPAC DIGITAL

Plan de respaldo y recuperacion

Como se respalda la base de datos de produccion, donde queda el respaldo, quien puede abrirlo y como se recupera la informacion cuando algo sale mal. Issue #914, requerimiento RNF-12.

Version	1.0
Fecha	11 de octubre de 2026
Aplica a	`Ecopac-Digital-Prod` (la base de produccion)
Workflow	`.github/workflows/respaldo-base.yml`
Scripts	`scripts/respaldo-base.sh`, `scripts/comprobar-restauracion.sh`

> Documento de uso interno de la organización. Describe cómo se protege y se recupera la información clínica; no contiene datos de pacientes ni credenciales.

Índice

Estado

1. Que se respalda y que no

2. Frecuencia, retencion y donde se guarda

3. Cuanto se puede perder y cuanto tarda recuperar

4. Quien tiene acceso y como se cifra

5. Procedimiento de recuperacion

    Paso 1. Bajar el respaldo

    Paso 2. Descifrar

    Paso 3. Comprobar que el archivo esta integro

    Paso 4. Restaurar en el stack local

    Paso 5. Comprobar que la restauracion es buena

    Paso 6. Llevarlo a produccion

    Paso 7. Cerrar

6. Calendario de pruebas de restauracion y su registro

7. Puesta en marcha (una sola vez)

    A. Generar el par de llaves (persona responsable de la base)

    B. Credencial de Google Drive (quien tenga la contrasena de la cuenta)

    C. Secrets de GitHub (equipo de desarrollo)

    D. Primera corrida y primera restauracion

Si el respaldo falla

Documentos relacionados

Estado

Este respaldo es el unico que existe. Produccion esta en el plan Free de Supabase, que no incluye respaldos (`docs/COSTOS-Y-LIMITES.md` , seccion 6.2). Si este workflow no corre, no hay ninguna copia de la base.

Pieza	Estado al 11 de octubre de 2026
Script y workflow de respaldo	Escritos
Volcado, cifrado, subida, comprobacion y retencion	**Ensayados** contra el stack local, con un destino local de rclone
Descifrado y restauracion	**Ensayados** contra el stack local: 96 tablas y 1195 filas identicas a las del respaldo
Cuenta de Google de la organizacion y credencial de Drive	**Pendiente.** El equipo solo tiene la direccion (`ecopac.digital@gmail.com`), no el acceso. Lo hace quien tenga la contrasena, con la seccion 7
Par de llaves de la organizacion	**Pendiente.** Lo genera la persona responsable de la base, con la seccion 7
Primera corrida en GitHub Actions	**Pendiente** de la #252: el workflow solo corre desde `main` y necesita los secrets de produccion
Primera restauracion desde un respaldo bajado de Drive	**Pendiente** de las tres filas anteriores. Produccion no recibe datos reales antes de hacerla

Ensayar contra el stack local prueba que los comandos, el orden y las comprobaciones son correctos. No prueba la credencial de Drive ni el volcado de un proyecto remoto: eso es lo que cierra la primera restauracion real.

1. Que se respalda y que no

Cada respaldo es **un solo archivo cifrado**, `respaldo-ecopac-prod-<fecha>T<hora>Z.tar.gz.gpg` , que por dentro trae:

Archivo	Que es	Para que sirve al recuperar
`datos.sql`	Todas las filas de `public` (pacientes, expedientes, consultas, recetas, inventario, jornadas, donaciones, bitacora) y de `auth` (las cuentas de usuario, con su contrasena cifrada)	Es lo que se restaura

Archivo	Que es	Para que sirve al recuperar
`esquema.sql`	La estructura de la base tal como estaba ese dia: tablas, funciones, politicas, triggers	Referencia. La estructura se reconstruye desde `supabase/migrations/`; este archivo sirve para detectar si la base y las migraciones no coinciden
`roles.sql`	Ajustes de los roles de la base	Referencia
`filas-por-tabla.tsv`	Cuantas filas traia cada tabla	Contra esto se comprueba la restauracion
`MANIFIESTO.txt`	Fecha, commit del repositorio, version de la CLI y huella SHA-256 de cada archivo	Saber sobre que version del esquema restaurar, y que el archivo no se dano

Lo que NO viaja en el respaldo, y donde esta cada cosa:

Que	Donde se recupera
Configuracion del Dashboard de Supabase (Site URL, redirect URLs, registro cerrado, SMTP, longitud de contrasena)	Se vuelve a configurar a mano con `docs/CONFIGURACION-SUPABASE.md`
Secretos de Vault y de las Edge Functions (SMTP, DSN de Sentry)	Se vuelven a cargar: `docs/CI-CD.md`, "Las notificaciones al administrador y su correo". Vault va cifrado con una llave del proyecto y no sirve en otro
Codigo, migraciones y Edge Functions	El repositorio de GitHub
Secrets de GitHub Actions	No se pueden leer una vez guardados. La lista con el origen de cada valor esta en `docs/CI-CD.md`, "Secrets"
Configuracion de Vercel y del dominio	`docs/QUICKSTART.md`
Archivos de Storage	El sistema no usa Storage (`docs/COSTOS-Y-LIMITES.md`, seccion 1)
La llave privada con la que se descifra	La guarda la persona responsable de la base, fuera de GitHub y fuera de Drive (seccion 4)

2. Frecuencia, retencion y donde se guarda

Frecuencia	**Todos los dias** a las 07:00 UTC (01:00 en Guatemala), mas a mano cuando haga falta
A mano	Actions > "Respaldo de la base" > Run workflow. Se hace antes de un despliegue grande o de una carga masiva

Retencion	Los **30** respaldos mas recientes. Al subir uno nuevo se borran los que sobran, sin pasar por la papelera
Donde	Google Drive de la cuenta de la organizacion (`ecopac.digital@gmail.com`), carpeta `respaldos-ecopac-digital`
Espacio	El respaldo del ensayo, con los datos de demostracion, pesa 132 KB. Treinta respaldos caben holgados en los 15 GB gratuitos; si la base crece hasta el limite del plan Free (500 MB), conviene revisar el espacio

La frecuencia es diaria, y no la semanal que la issue pedia como minimo, porque una jornada medica concentra en un dia el trabajo de semanas: perder la semana de una jornada es perder la jornada. El respaldo diario ademas cuenta como actividad y ayuda a que el proyecto no se pause.

La carpeta **la crea el workflow** en su primera corrida. No se crea a mano: la credencial solo ve los archivos que ella misma subio (seccion 4), y una carpeta creada desde el navegador seria invisible para el workflow.

3. Cuanto se puede perder y cuanto tarda recuperar

Perdida maxima de datos	**24 horas**: lo registrado desde el ultimo respaldo de la madrugada. Se reduce a cero sacando un respaldo a mano antes de una operacion riesgosa
Hasta donde se puede volver atras	30 dias
Tiempo de recuperacion	De 1 a 2 horas con la persona responsable disponible: bajar y descifrar (minutos), restaurar en local y comprobar (en el ensayo, menos de 15 minutos), y cargar en produccion

Lo que mas tarda en una recuperacion no es la tecnica: es conseguir a la persona que tiene la llave privada. Por eso la llave tiene dos copias (seccion 4).

4. Quien tiene acceso y como se cifra

Un respaldo de produccion contiene expedientes clinicos. El repositorio es publico.

Cifrado. El workflow cifra con la **llave publica** GPG de la organizacion antes de abrir la conexion con Drive, y borra los volcados sin cifrar antes de subir. Con una llave publica solo se puede cifrar. Para descifrar hace falta la **llave privada**, que no esta en GitHub, ni en Drive, ni en el repositorio.

Quien	Que puede hacer
El workflow de GitHub Actions	Crear un respaldo cifrado y subirlo; borrar los viejos. **No puede descifrar ninguno**
Quien administra los secrets del repositorio	Lo mismo que el workflow. Tampoco puede descifrar
Quien entra a la cuenta de Google de la organizacion	Ver, bajar y borrar los archivos cifrados. No puede leerlos sin la llave privada
La persona responsable de la base	La unica que tiene la llave privada y su frase de paso: la unica que puede abrir un respaldo

Persona responsable de la base: la designa la direccion de Ecopac y se anota aqui. Tiene que ser alguien de la organizacion, no del equipo de desarrollo, que se retira de todas las cuentas al terminar el soporte (#252, seccion 10).

Responsable	*pendiente de designar*
Suplente (tiene la segunda copia de la llave)	*pendiente de designar*

Reglas:

- La llave privada se guarda en **dos copias**: una en el gestor de contrasenas de la organizacion y otra fuera de linea (una memoria USB guardada bajo llave). Si se pierden las dos, **ningun respaldo se puede abrir** y no hay forma de recuperarlos.
- La frase de paso de la llave no se guarda junto a la llave.
- La carpeta de Drive no se comparte con nadie, ni por enlace.
- La credencial de Drive del workflow tiene el permiso `drive.file`: solo ve los archivos que ella misma creo, no el resto del Drive de la organizacion.
- Un respaldo descifrado se trabaja en un equipo de la organizacion y se borra al terminar. No se manda por correo ni por chat, ni se sube a ningun otro lado.
- Ningun respaldo queda como artefacto de GitHub Actions y ningun paso imprime su contenido.

5. Procedimiento de recuperacion

Lo hace la persona responsable de la base, con apoyo de quien de soporte tecnico. Hace falta un equipo con Git para Windows (trae `gpg` y `tar`), Docker Desktop, Node.js y el repositorio clonado. Los comandos se corren en **Git Bash**, desde la raiz del repositorio.

Antes de empezar, si la base de produccion todavia responde: sacar un respaldo a mano (seccion 2). Restaurar reemplaza todo lo que hay, y ese respaldo es la unica forma de volver atras.

Paso 1. Bajar el respaldo

Entrar a Google Drive con la cuenta de la organizacion, abrir la carpeta `respaldos-ecopac-digital` y descargar el archivo del dia al que se quiere volver. El nombre trae la fecha y la hora en UTC: `respaldo-ecopac-prod-20261012T070000Z.tar.gz.gpg` es del 12 de octubre a las 07:00 UTC (01:00 en Guatemala).

Paso 2. Descifrar

Con la llave privada ya importada en el equipo (`gpg --import respaldo-privada.asc` , una sola vez):

```
mkdir restauracion && cd restauracion
gpg --decrypt ../respaldo-ecopac-prod-20261012T070000Z.tar.gz.gpg | tar -xzf -
```

Pide la frase de paso de la llave. Quedan los cinco archivos de la seccion 1.

Paso 3. Comprobar que el archivo esta integro

```
sha256sum -c <(sed -n '/^Huellas/,$p' MANIFIESTO.txt | tail -n +2)
```

Las cuatro lineas tienen que decir `OK` . Si alguna falla, el archivo se dano: se usa el respaldo del dia anterior.

Paso 4. Restaurar en el stack local

Siempre primero en local, nunca directo sobre produccion: aqui se comprueba que el respaldo sirve sin arriesgar nada.

El esquema tiene que ser **el del dia del respaldo**. `MANIFIESTO.txt` dice de que commit salio:

```
cd ..
git checkout <commit que dice MANIFIESTO.txt>
npx supabase start
npx supabase db reset --no-seed
```

Vaciar lo que las migraciones siembran por su cuenta (catalogos y primer administrador), que chocaria con lo que trae el respaldo:

```
docker exec -i supabase_db_ecopac-digital psql -U postgres -d postgres -v ON_ERROR_STOP=1 <<'SQL'
DO $$
DECLARE tablas TEXT;
BEGIN
  SELECT string_agg(format('public.%I', tablename), ', ') INTO tablas
  FROM pg_tables WHERE schemaname = 'public';
  EXECUTE 'TRUNCATE ' || tablas || ', auth.users CASCADE';
END $$;
SQL
```

Cargar los datos, guardando la salida para revisarla:

```
docker exec -i supabase_db_ecopac-digital psql -U postgres -d postgres < restauracion/datos.sql >
restauracion/carga.log 2>&1
grep ERROR restauracion/carga.log
```

Dos errores son **esperados** y no invalidan nada: `permission denied for table buckets_vectors` y `... vector_indexes`, dos tablas vacias de Storage que el stack local no deja escribir. **Cualquier otro `ERROR` invalida la restauracion.** El porque del `--no-seed` y del `TRUNCATE` esta en `docs/CI-CD.md`, "Prueba de restauracion".

Un error `column "..." of relation "..." does not exist` significa que el esquema local no es el del dia del respaldo: se reviso el commit equivocado. Paso en el ensayo del 11 de octubre, con una base local que venia de una rama vieja: tres tablas quedaron vacias sin que la carga se detuviera. Por eso el paso 5 no es opcional.

Paso 5. Comprobar que la restauracion es buena

```
bash scripts/comprobar-restauracion.sh restauracion/filas-por-tabla.tsv
```

Compara, tabla por tabla, las filas restauradas con las que traia el respaldo. Tiene que terminar con `Restauracion comprobada: N tablas y M filas, identicas a las del respaldo`. Si lista alguna tabla, la restauracion no sirve.

Despues, dos comprobaciones mas:

1. Levantar la web contra el stack local (`npm run dev:web`), iniciar sesion con una cuenta real y abrir la ficha de un paciente con su historial.
2. `npx supabase test db`, que tiene que dar el mismo resultado que en una base recien creada.

Paso 6. Llevarlo a produccion

Este paso no se ha ensayado contra un proyecto remoto. Se ensaya en la primera restauracion real (seccion 6) y esta seccion se corrige con lo que se encuentre. Depende de que paso:

Que paso	Que se hace
Se borraron o danaron algunos registros	No se restaura produccion. Se buscan los registros en la copia local del paso 4 y se vuelven a cargar solo esos, con una migracion o desde la aplicacion. Asi no se pierde lo registrado despues del respaldo
La base quedo inservible, o hay que volver todo a un dia anterior	Se carga el respaldo completo sobre produccion, como abajo. **Se pierde todo lo registrado despues del respaldo**
El proyecto de Supabase se perdio	Se crea un proyecto nuevo (#252), se actualizan los secrets, el workflow aplica las migraciones, y despues se carga el respaldo como abajo. Ademas hay que rehacer lo de la tabla "Lo que NO viaja" de la seccion 1

Carga completa sobre produccion, con la cadena de conexion de Project Settings > Database y `psql` instalado. Se avisa antes a la organizacion para que nadie use el sistema mientras dura:

```
export ECOPAC_DB_URL="postgresql://postgres:<contrasena>@<host>:5432/postgres"
psql "$ECOPAC_DB_URL" -v ON_ERROR_STOP=1 -c "DO \$\$ DECLARE tablas TEXT; BEGIN SELECT
string_agg(format('public.%I', tablename), ', ') INTO tablas FROM pg_tables WHERE schemaname =
'public'; EXECUTE 'TRUNCATE ' || tablas || ', auth.users CASCADE'; END \$\$;"
psql "$ECOPAC_DB_URL" < restauracion/datos.sql > restauracion/carga-prod.log 2>&1
grep ERROR restauracion/carga-prod.log
bash scripts/comprobar-restauracion.sh restauracion/filas-por-tabla.tsv
```

Si el respaldo es anterior a la ultima migracion aplicada en produccion, la estructura de produccion ya no coincide con la del respaldo. En ese caso no se carga directo: se pide soporte tecnico (#921).

Paso 7. Cerrar

- Borrar la carpeta `restauracion/` y el archivo descargado.
- `npx supabase stop` y volver a la rama de trabajo (`git checkout develop`).
- Anotar lo ocurrido en la tabla "Registro" de `docs/CI-CD.md` .
- Entrar a la web de produccion y comprobar con la organizacion que la informacion esta.

6. Calendario de pruebas de restauracion y su registro

Un respaldo que nunca se restauro no es un respaldo.

Cuando	Que se hace	Quien
Antes de que produccion reciba datos reales	Primera restauracion completa desde un respaldo bajado de Drive (pasos 1 a 5 y 7)	Equipo de desarrollo con la persona responsable
Una vez al ano, al cierre de la temporada de jornadas	Lo mismo	Persona responsable, con soporte
Despues de cambiar `scripts/respaldo-base.sh`, el workflow o la version de la CLI de Supabase	Lo mismo	Quien haga el cambio
Una vez al mes	Revision rapida: que en Actions haya corridas en verde de los ultimos dias y que en Drive esten los archivos	Persona responsable

La revision mensual existe porque GitHub **desactiva los workflows programados de un repositorio sin actividad durante 60 dias**, sin avisar a la organizacion. Si eso pasa, se reactiva desde Actions > "Respaldo de la base" > Enable workflow.

Cada prueba se anota en la tabla "Registro" de `docs/CI-CD.md`, con fecha, resultado y quien la hizo.

Fecha	Que se probo	Resultado
11-10-2026	Ensayo completo contra el stack local (commit `7eedf18`, CLI 2.120.0, rclone 1.75.1 con destino local): respaldo, cifrado, subida, retencion, descifrado, restauracion y comprobacion	Respaldo de 132 KB con 96 tablas y 1195 filas. Huellas correctas. Carga con solo los dos errores esperados. `comprobar-restauracion.sh`: 96 tablas y 1195 filas identicas. `supabase test db`: 1211 de 1212, con el mismo fallo (`limites_de_uso.sql`, prueba 13) que da una base recien creada, o sea ajeno a la restauracion. Retencion: con tres respaldos viejos y `RESPALDO_CONSERVAR=2` borro los dos mas viejos y no toco archivos ajenos. Fallo de subida forzado: codigo 1 y ningun borrado
pendiente	Primera restauracion desde un respaldo bajado de Drive	-

7. Puesta en marcha (una sola vez)

Lo que falta para que el respaldo empiece a correr. Los pasos A y B los hace la organizacion; el equipo no tiene acceso a la cuenta de Google.

A. Generar el par de llaves (persona responsable de la base)

En Git Bash, en un equipo de la organizacion:

```
gpg --quick-generate-key "Respaldos Ecopac Digital <ecopac.digital@gmail.com>" rsa4096 encr never
gpg --armor --export ecopac.digital@gmail.com > respaldo-publica.asc
gpg --armor --export-secret-keys ecopac.digital@gmail.com > respaldo-privada.asc
```

El primer comando pide una frase de paso: larga, y anotada aparte de la llave. La llave no caduca a proposito: una llave vencida dejaria de cifrar y el respaldo empezaria a fallar.

`respaldo-privada.asc` se guarda en las dos copias de la seccion 4 y se borra del equipo. `respaldo-publica.asc` no es secreta: es la que va a GitHub.

B. Credencial de Google Drive (quien tenga la contrasena de la cuenta)

Pendiente de verificar con la cuenta real. Estos pasos siguen la documentacion de Google y de rclone, pero no se han podido ejecutar. Una cuenta de servicio no sirve: no tiene cuota de almacenamiento en un Gmail gratuito. Se usa OAuth con la propia cuenta.

1. Entrar a https://console.cloud.google.com con `ecopac.digital@gmail.com` y crear un proyecto (por ejemplo "Respaldos Ecopac Digital").
2. APIs y servicios > Biblioteca > habilitar **Google Drive API**.
3. Pantalla de consentimiento de OAuth: tipo **Externo**, y agregar el permiso `.../auth/drive.file`.
4. **Publicar la app** ("En produccion"). Es lo mas importante de esta lista: una app en modo "Prueba" entrega credenciales que **caducan a los 7 dias**, y el respaldo dejaria de funcionar una semana despues. `drive.file` no es un permiso sensible, asi que Google no pide verificacion.
5. Credenciales > Crear credenciales > ID de cliente de OAuth > tipo **App de escritorio**. Anotar el ID de cliente y el secreto.
6. En un equipo con rclone instalado, con la sesion de esa cuenta abierta en el navegador:

   ```
   rclone config create respaldo-ecopac drive client_id=<ID de cliente> client_secret=<secreto>
   scope=drive.file
   rclone config show respaldo-ecopac
   ```

El primer comando abre el navegador para autorizar. El segundo muestra una linea `token = {...}`: lo que va entre llaves, con las llaves, es el secret `RESPALDO_DRIVE_TOKEN`.

7. Borrar la credencial del equipo: `rclone config delete respaldo-ecopac`.

La credencial deja de servir si la organizacion revoca el acceso desde su cuenta de Google o si pasa seis meses sin usarse. Con un respaldo diario lo segundo no ocurre.

C. Secrets de GitHub (equipo de desarrollo)

```
gh secret set RESPALDO_GPG_CLAVE_PUBLICA < respaldo-publica.asc
gh secret set RESPALDO_DRIVE_CLIENT_ID
gh secret set RESPALDO_DRIVE_CLIENT_SECRET
gh secret set RESPALDO_DRIVE_TOKEN
```

Los tres ultimos piden el valor por teclado, para que no quede en el historial de la terminal. `SUPABASE_PROJECT_REF_PROD` y `SUPABASE_DB_PASSWORD_PROD` se crean con produccion (#252).

D. Primera corrida y primera restauracion

Con `develop` ya en `main` (#252): Actions > "Respaldo de la base" > Run workflow. Tiene que terminar en verde y el archivo tiene que aparecer en Drive. Con ese archivo se hace la primera restauracion (seccion 5) y se anota en las tablas de registro.

Si el respaldo falla

El workflow abre una issue titulada "El respaldo de produccion fallo" y la corrida queda en rojo. Que revisar, en orden:

Que dice la corrida	Causa mas probable	Que hacer
"Faltan los secrets"	Un secret no existe o se borro	Crearlo (seccion 7, paso C)
Falla "Vincular con Ecopac-Digital-Prod"	El proyecto esta pausado, o cambio la contrasena de la base	Reanudar el proyecto en el Dashboard, o actualizar `SUPABASE_DB_PASSWORD_PROD`
"El volcado ... salio vacio" o "no trae ninguna fila"	La CLI no pudo leer la base	Correrlo otra vez a mano; si se repite, revisar el estado del proyecto en Supabase
"contiene una llave PRIVADA"	Se guardo la llave equivocada en el secret	Esa llave ya no es segura: generar un par nuevo (paso A) y guardar la publica
Falla la subida con `invalid_grant` o `token expired`	La credencial de Drive caduco o fue revocada	Repetir el paso B desde el punto 6. Si caduco a los 7 dias, la app no estaba publicada (punto 4)
Falla la subida por espacio	El Drive se lleno	Liberar espacio en la cuenta o bajar `RESPALDO_CONSERVAR` en el workflow
No hay corridas recientes y nada fallo	GitHub desactivo el workflow por inactividad	Actions > "Respaldo de la base" > Enable workflow

Cuando falla la subida no se borra ningun respaldo anterior: la retencion solo corre despues de subir y comprobar el nuevo.

Para ensayar el script sin tocar produccion ni Drive, contra el stack local y con un destino en disco:

```
RESPALDO_ORIGEN=local RESPALDO_DESTINO=/ruta/a/una/carpeta \
  RESPALDO_CLAVE_PUBLICA="$(cat respaldo-publica.asc)" bash scripts/respaldo-base.sh
```

Documentos relacionados

- `docs/CI-CD.md` - "Respaldos y restauracion": el detalle del volcado y de la prueba, y la tabla "Registro"; "Secrets": la lista completa.
- `docs/CONFIGURACION-SUPABASE.md` - lo que se configura a mano en el Dashboard y no viaja en el respaldo.
- `docs/SEGURIDAD.md` y `docs/PROTECCION-DE-DATOS.md` - el tratamiento de los datos clinicos.
- `docs/COSTOS-Y-LIMITES.md` - por que produccion no tiene respaldos del proveedor.